

046882 HW/SW Co-design • Final project

Faster Python: nbody & raytrace

Profiling, software optimization and a hardware accelerator for pyperformance benchmarks

Student 1: Yousef Atrash • Student 2: Housin Mohammed Agbaria

1.60x

nbody, software optimization
identical results

2.10x

raytrace, software optimization
identical image

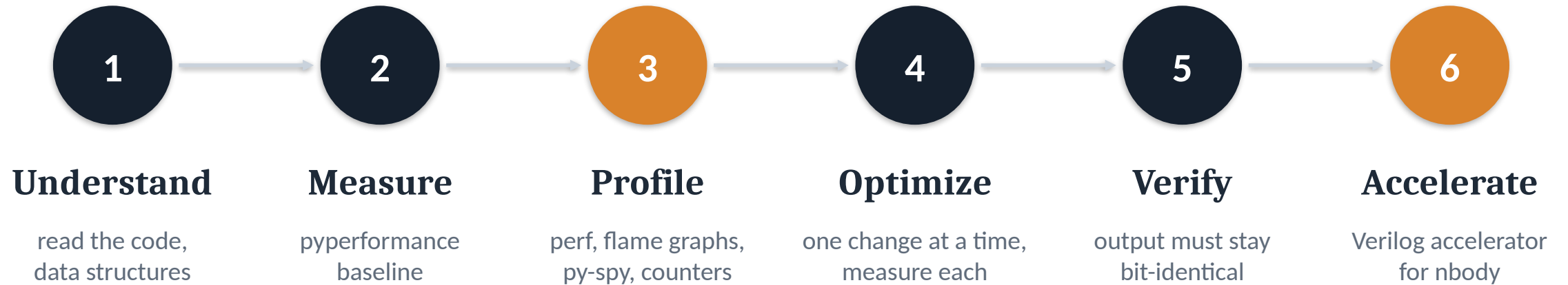
5.7x

nbody, hardware accelerator
(estimate, incl. PCIe overhead)

Draft: this presentation is not final and will be finalized before the presentation day.

How we worked

The talk follows the same order as the project



Rule we kept throughout: every change is its own variant, measured on its own with pyperformance --rigorous, and checked to give exactly the same output as the original.

1

Measuring on a noisy single-CPU VM

If the noise is bigger than the effect, the result means nothing

- **Environment:** course QEMU image, Ubuntu 22.04, 1 vCPU, Python 3.10.12, pyperformance 1.14.0
- **Timing:** pyperformance run --rigorous: 20 processes x 3 values = 120 samples per benchmark
- **Stability:** pyperf system tune (CPU pinning, IRQs moved away) before every timing session
- **Statistics:** median and MAD (robust to outliers), mean and std, histograms
- **Correctness:** verify scripts compare every result bit for bit with the original

raytrace baseline spread

±84 ms

first default run
(±10%, bigger than
the 7% target)

±19 ms

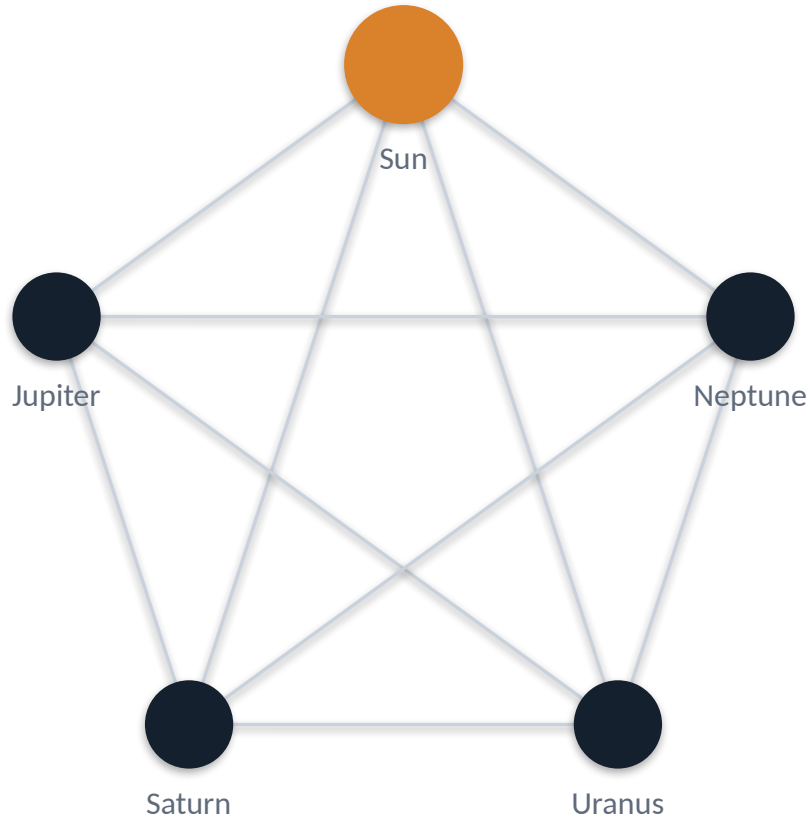
tuned + rigorous
(±2%, 120 samples)

The long tail of slow runs came from the VM being interrupted, not from the benchmark: the median barely moved (816 -> 809 ms), the tail disappeared.

2

nbody: planets under gravity

Sun + 4 giant planets, 20000 time steps per benchmark iteration



10 pairs per step · $O(N^2)$ pair loop

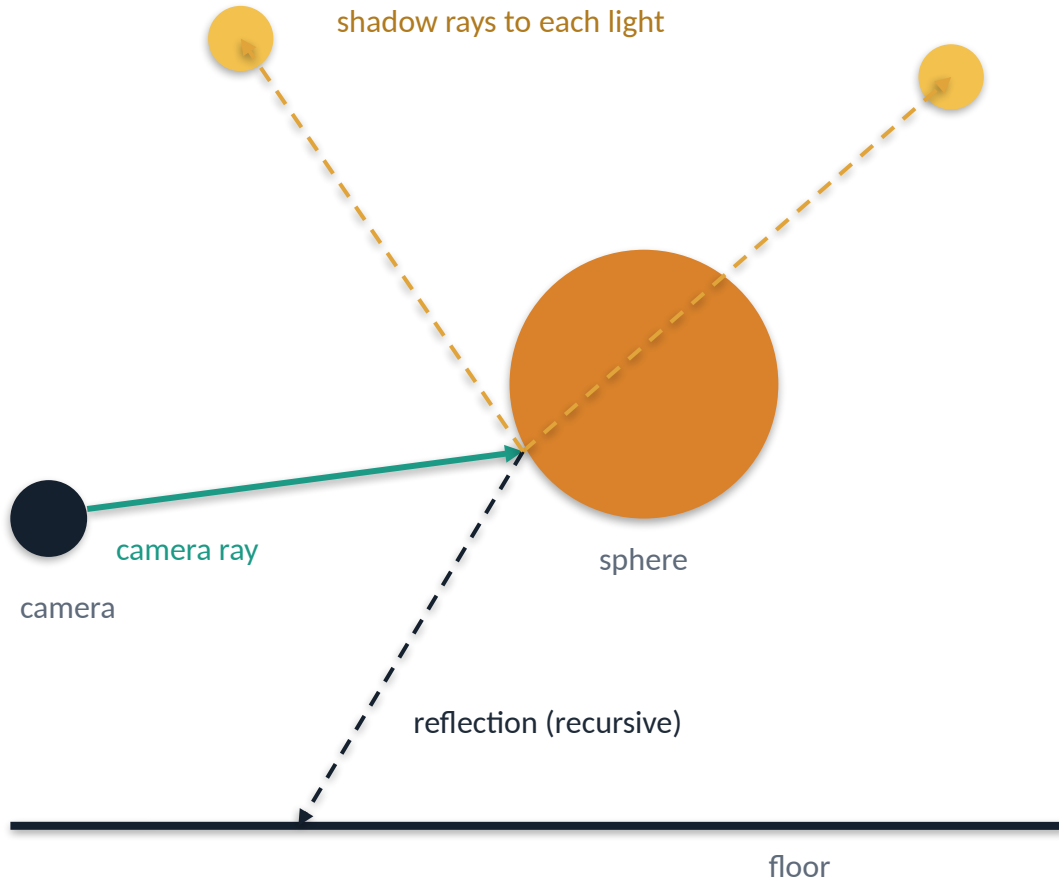
```
def advance(dt, n, bodies, pairs):
    for i in range(n):
        for (([x1,y1,z1], v1, m1),
              ([x2,y2,z2], v2, m2)) in pairs:
            dx = x1-x2; dy = y1-y2; dz = z1-z2
            mag = dt * (dx*dx+dy*dy+dz*dz)**(-1.5)
            v1[0] -= dx * m2 * mag # ... y, z
            v2[0] += dx * m1 * mag # ... y, z
        for (r, [vx, vy, vz], m) in bodies:
            r[0] += dt * vx # ... y, z
```

- **Data:** each body = ([x,y,z], [vx,vy,vz], mass), lists updated in place
- **Libraries:** none, plain Python floats (pyperf only for timing)

2

raytrace: a ray tracer in pure Python

100 x 100 image: 7 spheres, a checkerboard floor, 2 lights



- **Per pixel:** closest hit $\rightarrow$ colour = reflection + diffuse + ambient
- reflection: rayColour calls itself again (up to 4 levels deep)
- diffuse: one shadow ray per light, tested against every object
- **Data:** Vector and Point classes (x, y, z) with tiny methods: dot, scale, __sub__, plus no-op type checks (mustBeVector, isPoint)
- almost every arithmetic step creates a new Vector object
- **Libraries:** math, array (pixel buffer), pyperf

3

Our profiling toolbox

Flame graphs show WHERE the time goes; counters and the code show WHY

1

perf report

project guide command, python3-dbg

Which CPython C functions run: dispatch, list access, dict lookups, calls

2

Flame graph (pyperformance)

pyperformance --hook perf_record

The same, as a picture; only the timed benchmark code is recorded

3

Flame graph (py-spy)

samples Python's own frames

Python function names and source lines, with the call tree

4

perf stat

hardware counters, fixed work

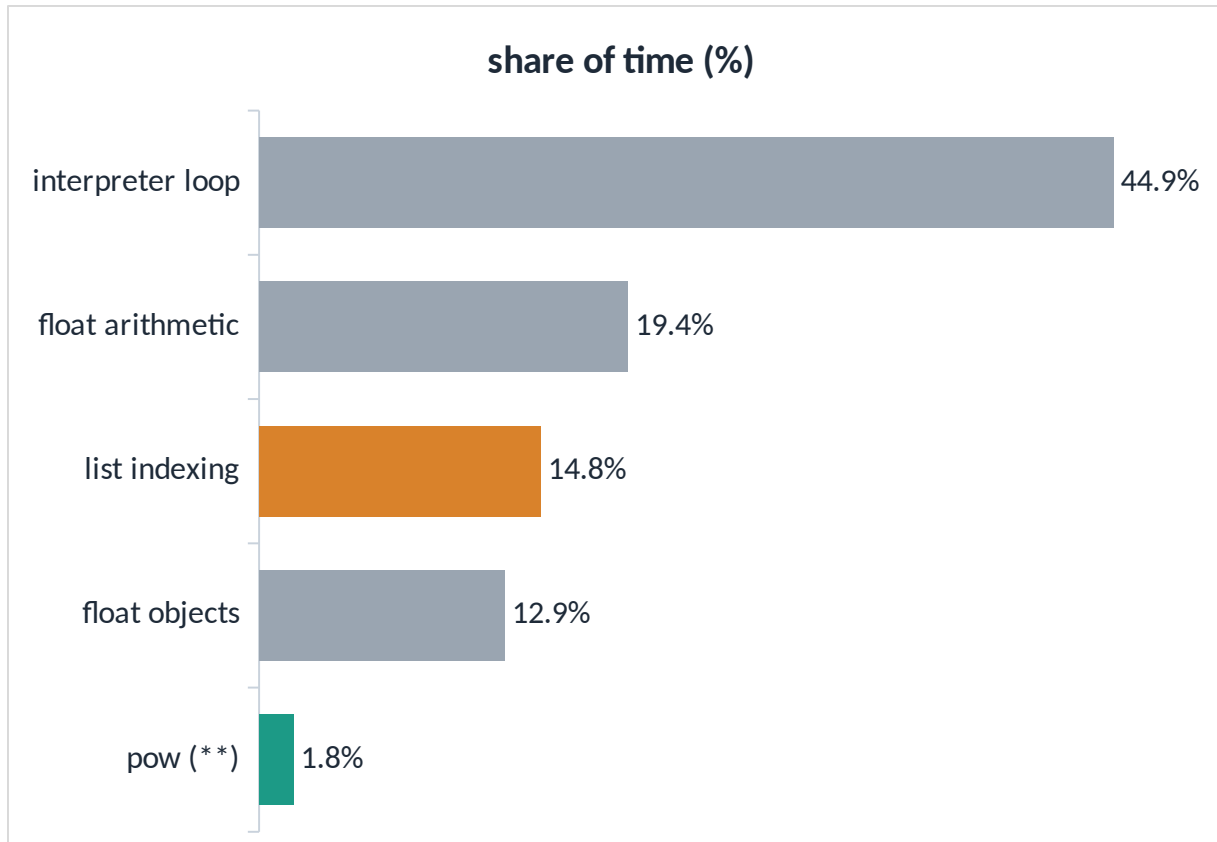
Instructions, IPC, cache misses, branch misses: memory or compute?

Plus cProfile for call counts, and small timeit/dis experiments to confirm a suspect before changing code. Python 3.10 has no frame pointers, so perf cannot name Python functions: that is why we added py-spy.

3

nbody: where the time goes

The math is cheap. The interpreter around it is not.



C-level flame graph (pyperformance), self time. Counters: IPC 2.94, cache misses 4.3%, branch misses 0.37%: not memory, not branches.

py-spy: hottest lines of advance()

48.3% `v1[0] -= dx*b2m ... v2[2] += dz*b1m`

18.3% `mag = dt * (d2) ** (-1.5)`

10.6% `r[0] += dt * vx ...`

8.4% `for ((x1,y1,z1),v1,m1), ... in pairs`

list read/write lines: ~59% of the run time

3

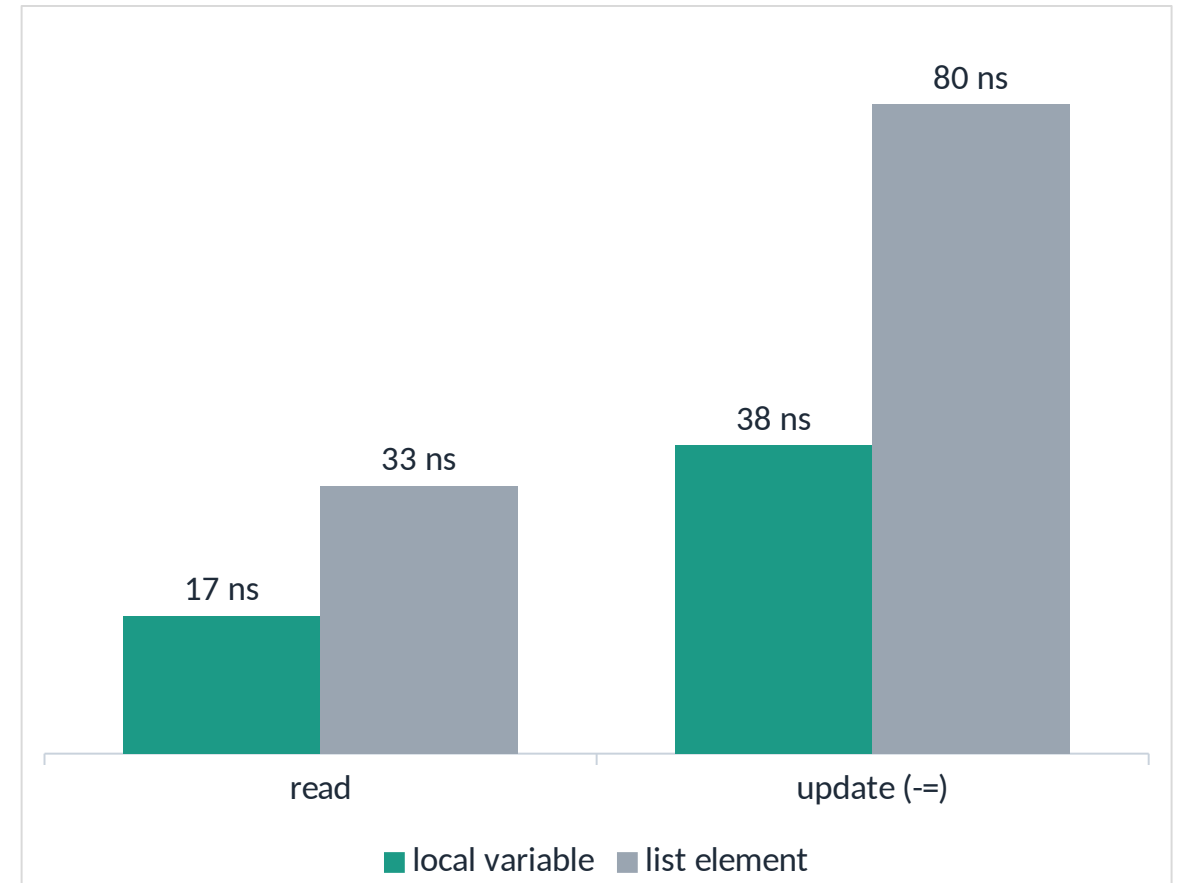
Why `v[0] -= x` is expensive

Checked with `dis` and `timeit` before changing any code

```
# local variable: x
LOAD_FAST    x

# list element: v[0]
LOAD_FAST    v
LOAD_CONST   0
BINARY_SUBSCR      # type check, index check
```

A list element costs extra bytecodes and C calls (type check, index conversion, fetch). `nbody` does 75 list reads and 75 list writes per step, 20000 steps per call.



timeit on the VM, nanoseconds per operation

4

nbody fix: keep the state in local variables

Copy once at the start of advance(), compute on locals, write back once at the end

```
(r0, v0, m0), ... (r4, v4, m4) = bodies
x0, y0, z0 = r0; vx0, vy0, vz0 = v0 # ...
for _ in range(n):
    # pair (0, 1)
    dx = x0 - x1; dy = y0 - y1; dz = z0 - z1
    mag = dt * ((dx*dx+dy*dy+dz*dz)**(-1.5))
    vx0 -= dx * (m1*mag) # locals only
    # ... 9 more pairs, then positions
r0[0] = x0; v0[0] = vx0 # write back once
```

Same operations in the same order, so the output is bit-for-bit identical (verify_nbody.py, 20000 steps). Other body counts fall back to the original loop.

1.60x

faster
230.4 -> 144.3 ms median

-34%

instructions
(perf stat, same work)

0%

list indexing left in the
flame graph (was 14.8%)

1.65x

second session:
reproducible

4

A change that did not help: `**` -> `sqrt`

`mag = dt / (d2 * sqrt(d2))` is mathematically the same as `dt * d2 ** (-1.5)`

1.8%

pow's share of the time:
the profile predicted
≤ 2% gain

1.01x

slower when measured
(and locals+sqrt slower
than locals alone)

n.s.

sqrt as a local variable:
143.3 vs 143.7 ms,
not significant

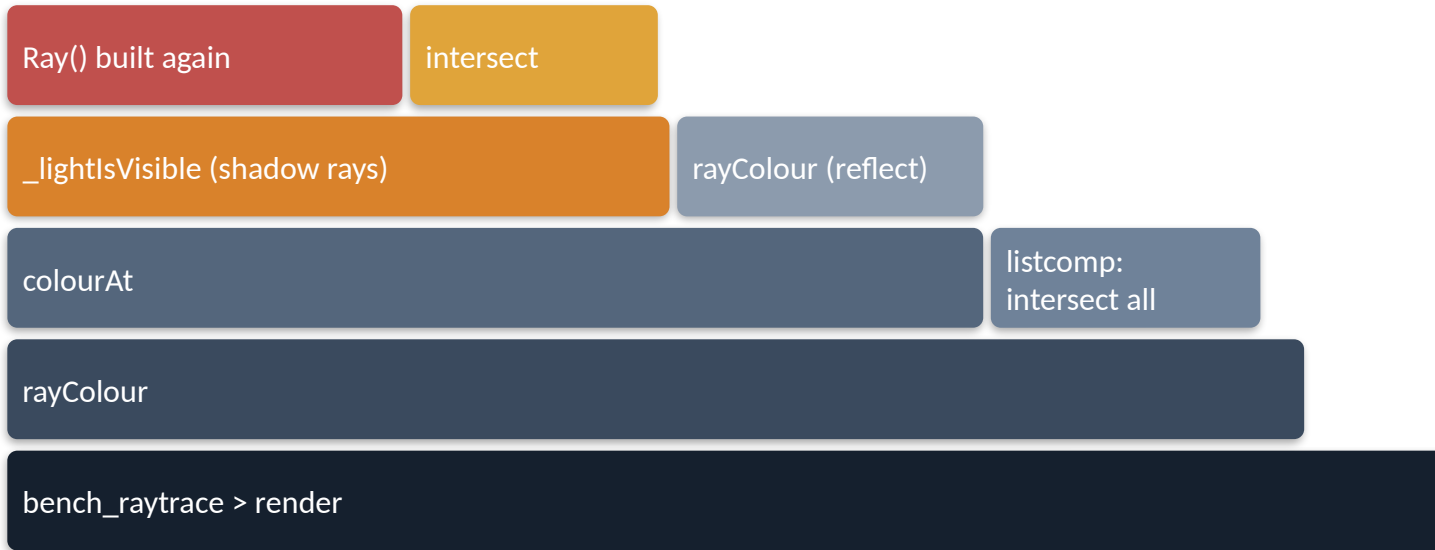
Why: in Python the cost is not the math. `d2 ** (-1.5)` is one bytecode that calls C pow directly (86 ns); `1/(d2*sqrt(d2))` needs a lookup of sqrt, a call, a multiply and a divide (93 ns).

Decision: dropped, kept `**`. The measurement stays in the report as a negative result that the profile correctly predicted.

3

raytrace: where the time goes

Python-level flame graph (py-spy): width = share of run time



intersectionTime from all callers: 43% (dot 20%, __sub__ 22%)

C level: function calls 19.8%, dict lookups 17.7%, math 0.1%

cProfile: 3.09 million Python calls and 453k new Vectors per image

Bottlenecks

- the same shadow ray rebuilt for every object (~28%)
- ~7 tiny method calls and a temporary Vector per ray-object test
- every Vector/Point has its own attribute dictionary

4

raytrace fixes: three targeted changes

Each measured on its own, each renders a bit-identical image

shadow_ray

Build Ray(p, l - p) once per light, not once per object

1.30x

sphere_floats

intersectionTime with plain floats: no temporary Vector, no dot/__sub__ calls

1.34x

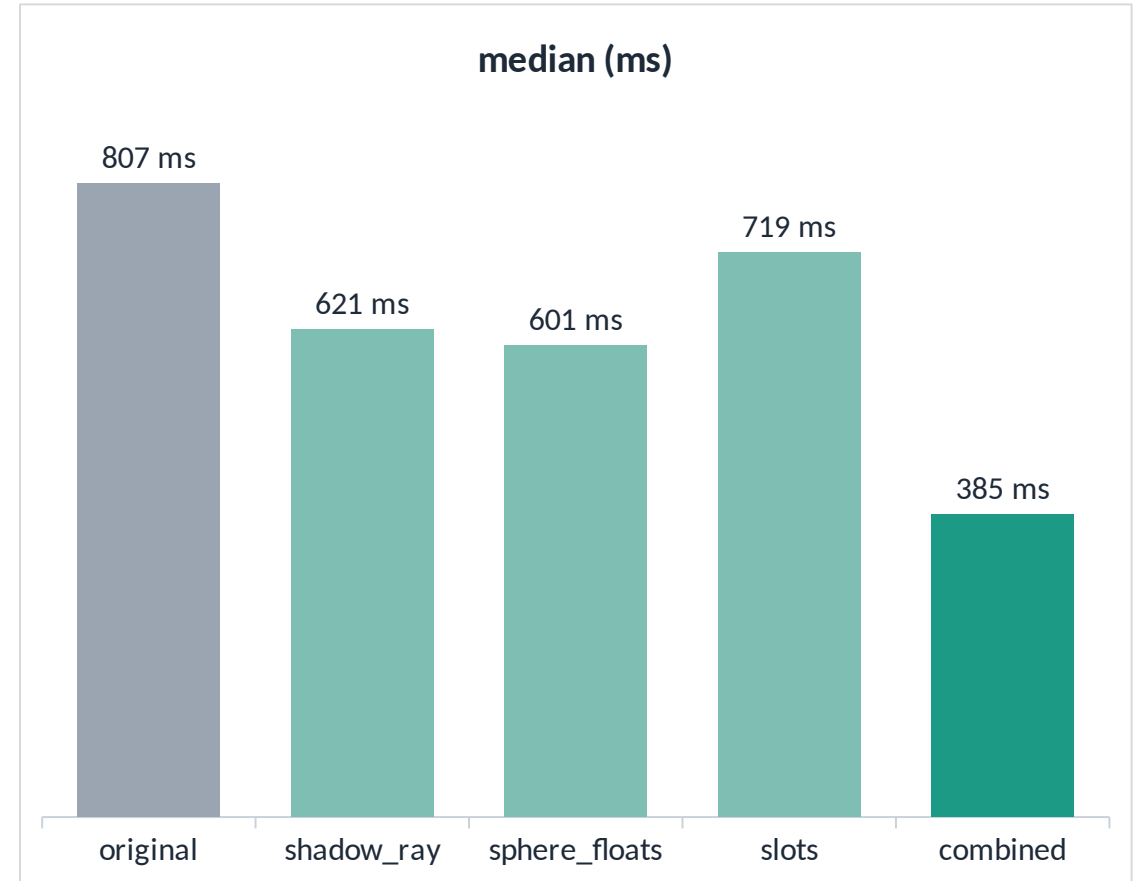
slots

__slots__ = ('x','y','z'): no per-object dictionary

1.12x

combined

2.10x

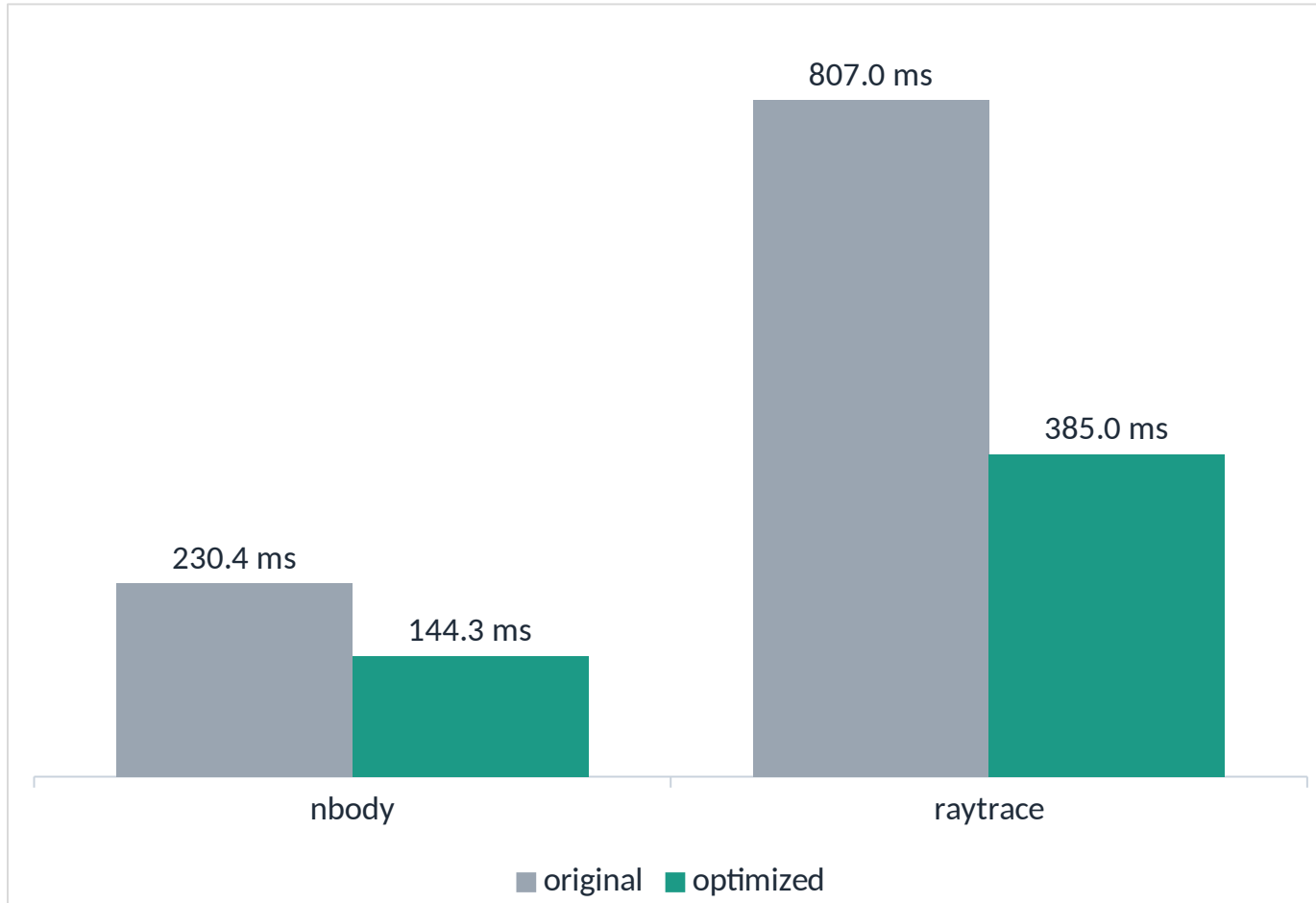


Savings overlap: 186 + 206 + 88 ms separately, but combined 385 ms, not 327 (fewer shadow rays also means fewer Vectors to speed up).

4

Software results

pyperformance --rigorous, 120 samples each, same tuned session



1.60x

nbody
37% less time

2.10x

raytrace
52% less time

-34%

nbody
instructions

-52%

raytrace
instructions

Target: 7% on two benchmarks.
Both outputs bit-identical.

5

Why a hardware accelerator, and why nbody

What is left after the software fix cannot be removed in Python

~100%

of nbody's time is one loop,
advance(): Amdahl is on our side

51%

of the optimized code is float
arithmetic + float objects:
only hardware removes that

280 B

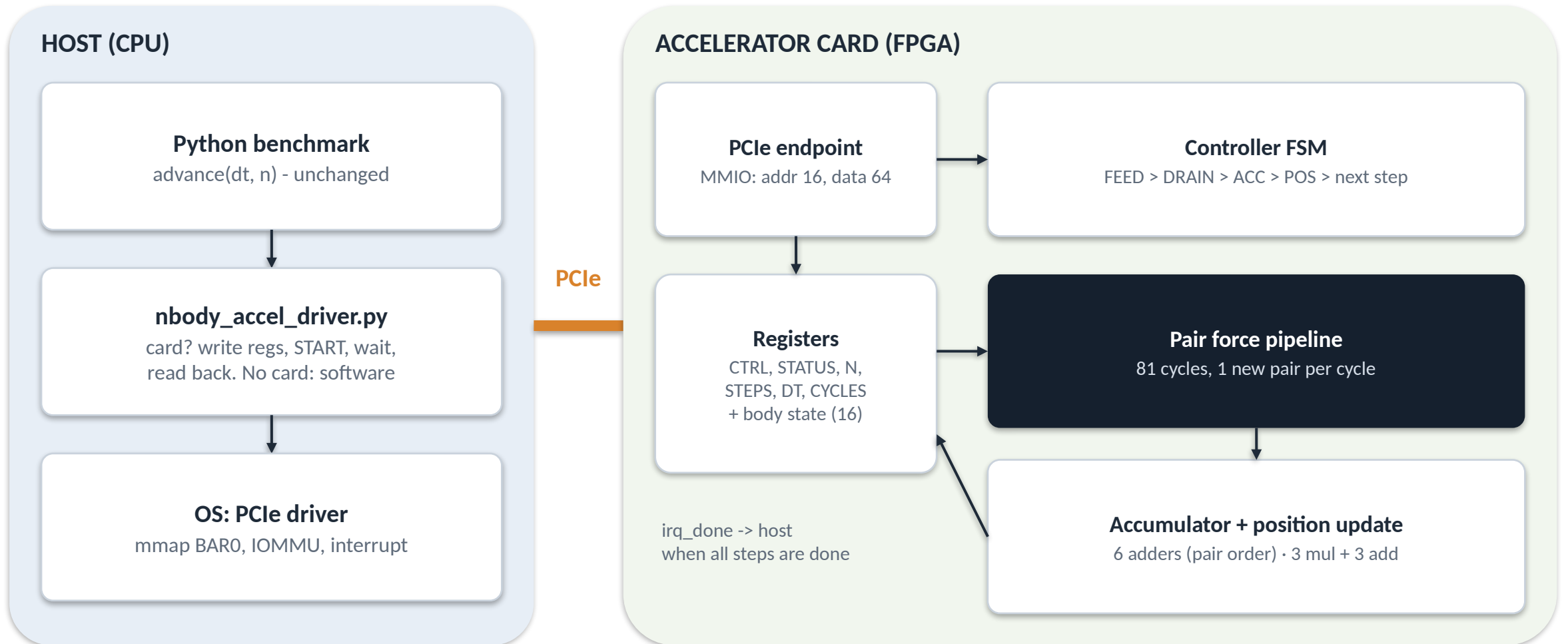
of state (5 bodies x 7 doubles):
one transfer in, one out
per call

raytrace instead? its math is only ~6% of the time; the cost is Python calls and objects. A ray-sphere unit would speed up that 6% and leave the rest in Python.

nbody: the whole advance() loop moves to hardware; 20000 steps run with no software in between.

The accelerator in the system

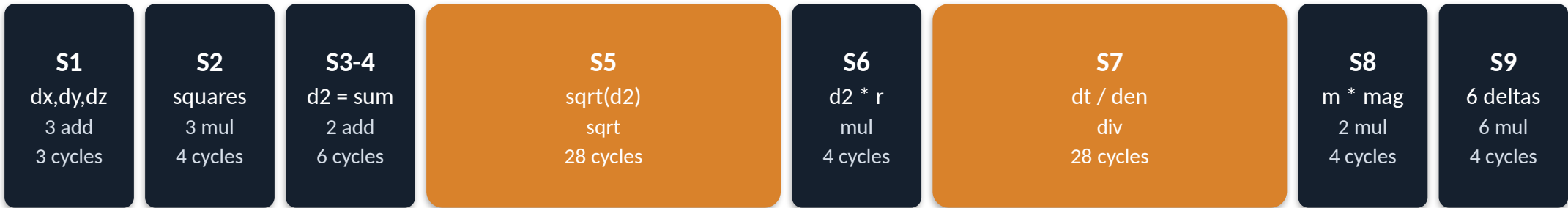
FPGA card on PCIe, 100 MHz, IEEE-754 double precision like Python



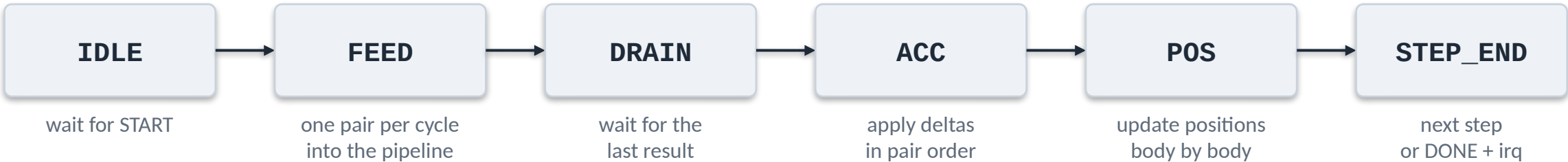
5

Inside: pair force pipeline and control

Pairs are independent within a step (forces depend only on positions)



81 cycles latency • div and sqrt: digit recurrence, 2 bits per stage, 28 stages each • all IEEE-754 double, round to nearest even



Measured cycles per step = 6P + 11N + 84 benchmark (N = 5, P = 10): 199 cycles = 1.99 μs at 100 MHz

How software talks to it

Memory-mapped registers in PCIe BAR0, 64-bit so one register holds one double

offset	register	meaning
0x0000	ID	"NBODYACC": driver detects the card
0x0008	CTRL	bit0 START
0x0010	STATUS	BUSY, DONE
0x0018	NUM_BODIES	2 .. 16
0x0020	NUM_STEPS	steps per call
0x0028	DT	IEEE-754 double
0x0030/38	STEPS_DONE, CYCLES	progress, performance counter
0x1000+	BODY[b] . f	x y z vx vy vz m

```
advance = accelerated(advance)
advance(0.01, 20000) # same call
```

- card present: write N, steps, dt, 35 body values; START; wait for DONE; read back into the same lists
- 70 register operations per call, independent of the number of steps
- no card, or > 16 bodies: the normal software advance() runs
- large systems: one DMA transfer instead of register writes

5

Verification: bit for bit

Icarus Verilog simulation, run_tests.sh rebuilds and reruns everything

274,014

floating-point test vectors:
add, mul, div, sqrt vs
Python's IEEE-754 results, 0
errors

4 / 4

system tests through the
registers: real benchmark
bodies and 2, 3, 16 bodies,
bit-identical

20,000

steps of the real benchmark
call: 3,980,000 cycles, all 35
values bit-identical

2 / 2

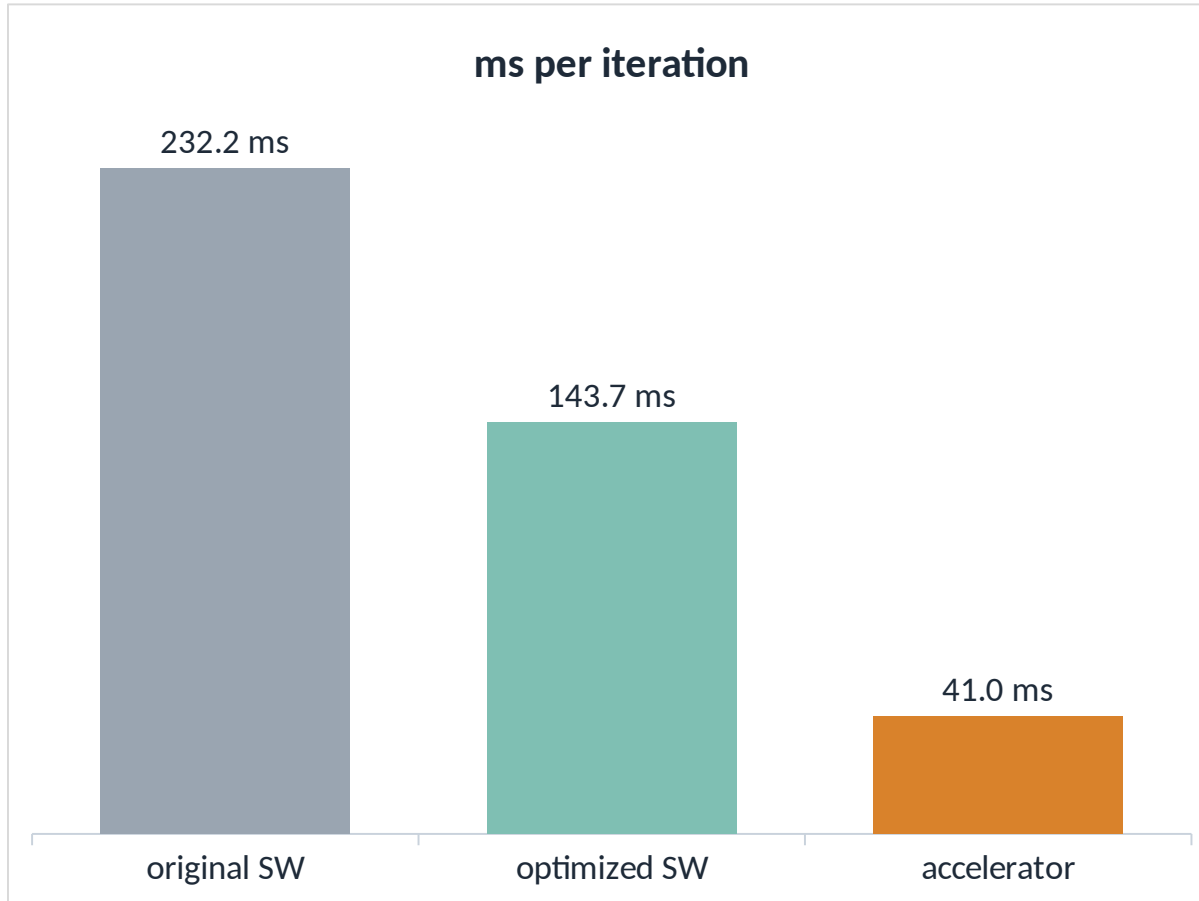
driver paths: software
fallback and offload, both
identical

Same results as the software: IEEE double with $1/(d2 \cdot \sqrt{d2})$, because +, -, x, / and sqrt have exactly one correct IEEE result and pow does not. Bit-identical to the software with the same formula; vs the original **: 1.3e-9 relative after 20000 steps (rounding only).

5

Estimated speedup (from measured numbers)

Software: pyperformance medians. Hardware: simulated cycles x 10 ns + PCIe overhead



accelerator = 20000 x 199 cycles x 10 ns (39.8 ms) + 70 PCIe accesses at 10 μ s + polling (1.2 ms)

5.7x

vs original software

3.5x

vs our optimized software

Break-even: the PCIe cost is paid once per call, so the card wins above ~125 steps (vs original) or ~230 (vs optimized). The benchmark does 20000.

More bodies, bigger win: 1.3x at 2, 3.6x at 5, 5.1x at 8, 6.6x at 16 bodies.

Assumptions: 100 MHz reached (not synthesized), 10 μ s per register access (pessimistic), FPGA configured once.

Design trade-offs

Performance, area, frequency and power

IEEE double vs fixed point

Exact same results as the software. Fixed point is smaller and uses less power, but drifts over 20000 steps.

Div/sqrt: bits per stage

2 bits at 100 MHz is our balance. 1 bit: ~150 MHz, 2x registers, more power. 4 bits: smaller, but ~50 MHz.

Accumulation order

Must follow pair order to stay exact: float addition is not associative.

Parallelism vs area

Parallel position update: -20% cycles for 5x that area. A second pipeline: latency, not throughput, is the limit.

MMIO vs DMA

Registers for small systems (70 accesses). DMA of one buffer for large systems, with IOMMU setup.

Power and energy

CPU busy 232 ms vs card done in 41 ms: energy per run drops, if the FPGA is configured once and stays ready.

Conclusions

1

Measure before you trust

Tuned VM, 120 samples, medians: the noise was bigger than the target at first.

2

Python's cost is the interpreter

Not the math: list access, calls, objects, dictionaries.

3

Profile, then change one thing

Every fix came from a flame graph, and the profile even predicted the one that failed.

4

Offload the whole loop

Hardware wins on big batches: 20000 steps per PCIe round trip.

1.60x / 2.10x

software: nbody / raytrace,
identical outputs

5.7x

nbody accelerator (estimate),
verified bit for bit in simulation

Thank you. Questions?

Demo: everything is reproducible

All in the Git repository

```
# software: setup, verify, profile, benchmark, compare
sudo ./script_nbody.sh
sudo ./script_raytrace.sh
sudo SKIP_SETUP=1 FAST=1 ./script_nbody.sh    # quick

# correctness only
python3 nbody/verify_nbody.py
python3 raytrace/verify_raytrace.py

# hardware: FP units, accelerator, driver
hw/nbody_accel/run_tests.sh
python3 hw/nbody_accel/model/estimate_speedup.py
```

- **Reports:** report_nbody.txt, report_raytrace.txt
- **Flame graphs:** results/profiling/<bench>/<version>/
- **Raw timings:** results/*.json + commands
- **Hardware:** hw/nbody_accel/ (docs, rtl, tb, sw)
- **AI prompts:** prompt.txt